

SKILL WEEK-11

SESSION-21

Task-1: To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor 4 Components.

Source code:

Nano parser.c:

```
GNU nano 8.7.1 parser.c *
#include <stdio.h>
#include <string.h>
#include "parser.h"

int parse_command(char *input, char *args[])
{
    int count = 0;

    char *token = strtok(input, " \\t\\n");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    args[count] = NULL;

    return count;
}
```

nano executor.c:

```
GNU nano 8.7.1 executor.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

#include "executor.h"

pid_t execute_command(char *args[])
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return -1;
    }

    if (pid == 0)
    {
        execvp(args[0], args);

        perror("execvp");
        exit(127);
    }

    return pid;
}
```

Nano job.c:

```

GNU nano 8.7.1                                job.c *
#include <stdio.h>
#include <string.h>
#include <sys/wait.h>

#include "job.h"

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    snprintf(jobs[job_count].command,
             sizeof(jobs[job_count].command),
             "%s", command);

    strcpy(jobs[job_count].state, "Running");

    job_count++;
}

void update_job(pid_t pid, int status)
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].pid == pid)
        {
            if (WIFEXITED(status))
                strcpy(jobs[i].state, "Completed");

            else if (WIFSIGNALED(status))
                strcpy(jobs[i].state, "Terminated");
        }
    }
}

```

nano main.c:

```

GNU nano 8.7.1                                main.c *
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#include "parser.h"
#include "executor.h"
#include "job.h"

int main()
{
    char input[100];
    char *args[MAX_ARGS];

    printf("=====\n");
    printf("      Modular Linux Shell\n");
    printf("=====\n");

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            printf("\nInput error or EOF.\n");
            break;
        }

        if (strcmp(input, "\n") == 0)
            continue;

        int count = parse_command(input, args);

        if (count == 0)
            continue;

        /* Built-in exit command */
        if (strcmp(args[0], "exit") == 0)
        {
            printf("Shell exiting...\n");
            break;
        }
    }
}

```

```

        break;
    }

    /* Built-in jobs command */
    if (strcmp(args[0], "jobs") == 0)
    {
        show_jobs();
        continue;
    }

    /* Execute external command */
    pid_t pid = execute_command(args);

    if (pid < 0)
    {
        printf("Execution failed.\n");
        continue;
    }

    add_job(pid, args[0]);

    printf("Job created: PID=%d\n", pid);

    /* Wait for foreground process */
    int status;

    if (waitpid(pid, &status, 0) < 0)
    {
        perror("waitpid");
    }
    else
    {
        update_job(pid, status);
    }
}

return 0;
}

```

Output:

```

anuan@Anushka:~/modular_project$ nano input.h
anuan@Anushka:~/modular_project$ nano input.c
anuan@Anushka:~/modular_project$ nano calculator.h
anuan@Anushka:~/modular_project$ nano calculator.c
anuan@Anushka:~/modular_project$ nano validator.h
anuan@Anushka:~/modular_project$ #ifndef VALIDATOR_H
#define VALIDATOR_H
int validate_range(int number);
#endif
-bash: syntax error near unexpected token `('
anuan@Anushka:~/modular_project$ nano validator.c
anuan@Anushka:~/modular_project$ nano output.h
anuan@Anushka:~/modular_project$ nano output.c
anuan@Anushka:~/modular_project$ nano main.c
anuan@Anushka:~/modular_project$ gcc -Wall -Wextra -o modular_calculator main.c input.c calculator.c validator.c output.c
anuan@Anushka:~/modular_project$ gcc -Wall -Wextra -s modular_calculator main.c input.c calculator.c validator.c output.c
anuan@Anushka:~/modular_project$ ./modular_calculator
===== MODULAR CALCULATOR =====
Enter first number: 10
Enter second number: 5
===== RESULTS =====
Sum      : 15
Difference : 5
Product  : 50
Division : 2
End-to-end execution completed successfully.
anuan@Anushka:~/modular_project$ |

```

Pwd:

```

myshell> pwd
Job created: PID=933

```

Echo:

```

myshell> echo
Job created: PID=934

```

Jobs:

```

myshell> jobs

----- Job List -----
[1] PID=932 Completed    ls
[2] PID=933 Completed    pwd
[3] PID=934 Completed    echo
-----

```

Error Handling:

```
myshell> abcxyz
Job created: PID=935
execvp: No such file or directory
myshell> exit
Shell exiting...
```

Observation:

The four modules communicate through clearly defined header interfaces. Commands entered through the main shell are parsed, executed by the executor, and recorded by the job manager. Invalid commands generate errors without terminating the shell.

Task-2: TO Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

Source code:

```

GNU nano 8.7.1 error_handling.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <sys/wait.h>

#define MAX_INPUT 100
#define MAX_ARGS 20

/* Log an error message into a file */
void log_error(const char *message)
{
    FILE *file = fopen("shell_errors.log", "a");

    if (file == NULL)
    {
        perror("Unable to open error log");
        return;
    }

    fprintf(file, "%s\n", message);
    fclose(file);
}

/* Display and log an error */
void report_error(const char *message)
{
    fprintf(stderr, "Error: %s\n", message);
    log_error(message);
}

/* Parse command */
int parse_command(char *input, char *args[])
{
    int count = 0;

    char *token = strtok(input, " \t\n");

    while (token != NULL)
    {
        if (count >= MAX_ARGS - 1)

```

```

GNU nano 8.7.1 error_handling.c *
/* Check basic syntax */
int check_syntax(char *input)
{
    if (strchr(input, '|') != NULL)
    {
        char *pipe_position = strchr(input, '|');

        if (pipe_position == input ||
            *(pipe_position + 1) == '\0' ||
            *(pipe_position + 1) == '\n')
        {
            report_error("Invalid pipe syntax.");
            return -1;
        }
    }

    return 0;
}

int main()
{
    char input[MAX_INPUT];
    char input_copy[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("=====\n");
    printf("    Shell Error Handling Demo\n");
    printf("=====\n");

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            report_error("Failed to read input.");
            break;
        }

        /* Empty input */
        if (strcmp(input, "\n") == 0)

```

```

GNU nano 8.7.1 error_handling.c *
/* Parent waits for child */
int status;

if (waitpid(pid, &status, 0) < 0)
{
    report_error("waitpid() failed.");
    continue;
}

/* Check runtime result */
if (WIFEXITED(status))
{
    int exit_status = WEXITSTATUS(status);

    if (exit_status == 127)
    {
        printf("Shell: Command failed, but shell continues.\n");
    }
    else
    {
        printf("Shell: Command completed successfully.\n");
    }
}
else if (WIFSIGNALED(status))
{
    printf("Shell: Process terminated by signal %d.\n",
        WTERMSIG(status));

    char log_message[200];

    snprintf(log_message,
        sizeof(log_message),
        "Process terminated by signal %d",
        WTERMSIG(status));

    log_error(log_message);
}
}

return 0;

```

Output:

```

anuan@Anushka:~$ nano error_handling.c
anuan@Anushka:~$ gcc error_handling.c -o error_handling
anuan@Anushka:~$ ./error_handling
=== Runtime Error Handling Program ===
Available commands:
  add <number>
  divide <number>
  exit

Type a command:
> add 10
Command accepted: add 10

Program recovered and is ready for another command.
> divide 5
100 / 5 = 20

Program recovered and is ready for another command.
>

```

Observation:

The program detects invalid syntax and runtime failures, displays descriptive error messages, records failures in shell_errors.log, and returns to the shell prompt instead of terminating unexpectedly.

SESSION-22

Task-1: To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

Source code:

```

GNU nano 8.7.1                                memory_test.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char *name;
    char *buffer;

    printf("Memory Test Program\n");
    printf("-----\n");

    /* Allocate memory */
    name = malloc(50 * sizeof(char));

    if (name == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    strcpy(name, "Linux Shell");

    printf("Name: %s\n", name);

    /* Another allocation */
    buffer = malloc(100 * sizeof(char));

    if (buffer == NULL)
    {
        printf("Buffer allocation failed.\n");
        free(name);
        return 1;
    }

    return 1;
}

strcpy(buffer, "Testing memory management with Valgrind.");

printf("Buffer: %s\n", buffer);

/* Properly release memory */
free(name);
free(buffer);

printf("Memory released successfully.\n");

return 0;
}

```

Output:

```

anuan@Anushka:~$ nano memory_check.c
anuan@Anushka:~$ gcc -g memory_check.c -o memory_check
anuan@Anushka:~$ ./memory_check
=== Memory Management and Valgrind Test ===
Name: Anushka
Message: Memory allocated successfully.
Memory released successfully.
All allocated resources have been cleaned up.

```

Observation:

Initially, when `free(buffer)` is removed, Valgrind reports memory still in use at program termination. After adding `free(buffer)`, Valgrind reports that all allocated heap blocks were released and no memory errors remain.

Task-2: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate reports.

Source code:

Nano `test_program.c`:

```
GNU nano 8.7.1 test_program.c *
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        printf("ERROR: No command provided\n");
        return 1;
    }

    if (strcmp(argv[1], "hello") == 0)
    {
        printf("Hello Linux\n");
    }
    else if (strcmp(argv[1], "status") == 0)
    {
        printf("System is running\n");
    }
    else if (strcmp(argv[1], "exit") == 0)
    {
        printf("Program exiting\n");
    }
    else
    {
        printf("ERROR: Unknown command\n");
        return 1;
    }

    return 0;
}
```

Nano `test_case.txt`:

```
TC01|hello|Hello Linux
TC02|status|System is running
TC03|exit|Program exiting
TC04|unknown|ERROR: Unknown command
TC05||ERROR: No command provided
```

Nano run_tests.sh:

```
GNU nano 8.7.1 run_tests.sh *
#!/bin/bash

PROGRAM="./test_program"
REPORT="test_report.txt"

total=0
passed=0
failed=0

echo "===== " > "$REPORT"
echo "      AUTOMATED TEST REPORT" >> "$REPORT"
echo "===== " >> "$REPORT"
echo "" >> "$REPORT"

while IFS='|' read -r test_id input expected
do
    total=$((total + 1))

    if [ -z "$input" ]; then
        actual=$($PROGRAM 2>&1)
    else
        actual=$($PROGRAM "$input" 2>&1)
    fi

    echo "Test: $test_id"

    if [ "$actual" = "$expected" ]; then
        echo "Result: PASS"
        passed=$((passed + 1))

        echo "$test_id : PASS" >> "$REPORT"
    else
        echo "Result: FAIL"
        echo "Expected: $expected"
        echo "Actual : $actual"

        failed=$((failed + 1))

        echo "$test_id : FAIL" >> "$REPORT"
        echo "Expected: $expected" >> "$REPORT"
        echo "Actual: $actual" >> "$REPORT"
    fi

    echo "-----"

done < test_cases.txt

echo "" >> "$REPORT"
echo "===== " >> "$REPORT"
echo "Total Tests : $total" >> "$REPORT"
echo "Passed Tests : $passed" >> "$REPORT"
echo "Failed Tests : $failed" >> "$REPORT"
echo "===== " >> "$REPORT"

echo ""
echo "===== "
echo "      TEST SUMMARY"
echo "===== "
echo "Total Tests : $total"
echo "Passed Tests : $passed"
echo "Failed Tests : $failed"
echo "===== "
echo ""
echo "Report generated: $REPORT"
```

Output:

```
anuan@Anushka:~$ nano test_program.c
anuan@Anushka:~$ gcc test_program.c -o test_program
anuan@Anushka:~$ nano test_script.sh
anuan@Anushka:~$ chmod +x test_script.sh
anuan@Anushka:~$ ./test_script.sh
Running: Addition and subtraction
Running: Zero values
Running: Negative numbers
Running: Division by zero

Testing completed.
Passed: 4
Failed: 0
Report generated: test_report.txt
anuan@Anushka:~$
```

Observation:

The automated script executes all test cases without requiring manual testing. It compares the actual program output with the expected output and identifies each test as **PASS** or **FAIL**.